

Reviewer Pack — Minimal Index of Tropicalized Quivers vs Resolution Proof Tr...

Entry 2d8adf396e64 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 07:43:17 UTC

Minimal Index of Tropicalized Quivers vs Resolution Proof Tree Diameter

Entry ID: 2d8adf396e64 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 07:43:17 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory

Statement:

[‘For a given satisfiability problem represented as a Boolean circuit, the minimal index of its tropicalized quiver is equal to the minimum diameter of any resolution proof tree for that circuit.’, ‘Equivalently, if an instance of size n requires a resolution proof tree with a diameter greater than d , then its tropicalized quiver has a minimal index greater than or equal to d .’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a way to encode Boolean circuits in terms of quivers, which are directed graphs. The minimal index of a quiver is related to the complexity of certain algebraic operations on it. This conjecture suggests that this algebraic property could be directly connected to the complexity of resolution proofs for circuits.’, ‘If true, this would provide a new insight into the structure of resolution proof trees and potentially lead to more efficient algorithms for proving satisfiability.’]

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f4a108c15a947b8d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all circuits of size $n \leq 40$, the minimal index of the tropicalized quiver is equal to the minimum diameter of any resolution proof tree with a difference of at most 0.5 units.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND resolution proof tree diameter IN complexity theory - Boolean circuit AND minimal index OF tropicalized quiver AND resolution proof tree diameter - satisfiability problem AND tropicalization AND quiver index = resolution proof tree diameter

Top relevant hits considered: - [<http://arxiv.org/abs/1206.1925v1>] Counting Algebraic Curves with Tropical Geometry - [<http://arxiv.org/abs/1610.00298v4>] Khovanskii bases, higher rank valuations and tropical geometry - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmuller and Siegel spaces - [<http://arxiv.org/abs/1102.1242v2>] A refinement of Stone duality to skew Boolean algebras - [<http://arxiv.org/abs/2207.08701v3>] Notes on Boolean Read-k and Multilinear Circuits - [<http://arxiv.org/abs/0903.3848v1>] Join-irreducible Boolean functions - [<http://arxiv.org/abs/math/0605345v3>] A tropical approach to secant dimensions - [<http://arxiv.org/abs/2306.17566v2>] Imputing phylogenetic trees using tropical polytopes over the space of phylogenetic trees

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def generate_circuit(n):
    if n == 1:
        return "A"
    else:
        left = generate_circuit(n // 2)
        right = generate_circuit(n - n // 2)
        return f"({left} OR {right})"

def tropicalize_quiver(circuit):
    if circuit.isalpha():
        return {circuit: set()}
    elif "OR" in circuit:
        left, right = circuit.split(" OR ")
        quiver_left = tropicalize_quiver(left)
        quiver_right = tropicalize_quiver(right)
        quiver = {}
```

```

    for u in quiver_left:
        quiver[u] = quiver_left[u]
    for v in quiver_right:
        if v not in quiver:
            quiver[v] = set()
            quiver[v].update(quiver_right[v])
    return quiver
elif "AND" in circuit:
    left, right = circuit.split(" AND ")
    quiver_left = tropicalize_quiver(left)
    quiver_right = tropicalize_quiver(right)
    quiver = {}
    for u in quiver_left:
        if u not in quiver:
            quiver[u] = set()
            quiver[u].update(quiver_left[u])
    for v in quiver_right:
        quiver[v] = quiver_right[v]
    return quiver
else:
    raise ValueError("Invalid circuit")

def compute_minimal_index(quiver):
    def dfs(node, visited, index):
        if node in visited:
            return index
        visited.add(node)
        max_index = 0
        for neighbor in quiver[node]:
            max_index = max(max_index, dfs(neighbor, visited, index + 1))
        return max_index

    min_index = float('inf')
    for node in quiver:
        min_index = min(min_index, dfs(node, set(), 0))
    return min_index

def compute_resolution_diameter(circuit):
    def resolve(expr):
        if expr.isalpha():
            return {expr}
        elif "OR" in expr:
            left, right = expr.split(" OR ")
            return resolve(left).union(resolve(right))
        elif "AND" in expr:
            left, right = expr.split(" AND ")
            return resolve(left).intersection(resolve(right))
        else:
            raise ValueError("Invalid circuit")

    def dfs(node, visited):
        if node in visited:
            return 0
        visited.add(node)
        max_depth = 0
        for neighbor in quiver[node]:

```

```

        max_depth = max(max_depth, dfs(neighbor, visited))
    return max_depth + 1

quiver = tropicalize_quiver(circuit)
resolution_set = resolve(circuit)
diameter = float('-inf')
for node in resolution_set:
    diameter = max(diameter, dfs(node, set()))
return diameter

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    circuit = generate_circuit(n)

    try:
        quiver = tropicalize_quiver(circuit)
        minimal_index = compute_minimal_index(quiver)
        resolution_diameter = compute_resolution_diameter(circuit)

        return {
            "metric_name": "Minimal Index vs Resolution Diameter",
            "metric_value": abs(minimal_index - resolution_diameter),
            "instances_tested": 1,
            "conjecture_holds": abs(minimal_index - resolution_diameter) <= 0.5,
            "counterexample": ""
        }
    except Exception as e:
        return {
            "metric_name": "Minimal Index vs Resolution Diameter",
            "metric_value": float('nan'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": str(e)
        }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        result = f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}"
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        result = f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}"
    else:
        result = "RESULT: INCONCLUSIVE reason=unreachable"

```

```
print(result)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
_name': 'Minimal Index vs Resolution Diameter', 'metric_value': nan, 'instances_tested': 1, 'conjecture': 'Minimal Index vs Resolution Diameter'
TRIAL: {'metric_name': 'Minimal Index vs Resolution Diameter', 'metric_value': nan, 'instances_tested': 1, 'conjecture': 'Minimal Index vs Resolution Diameter'
TRIAL: {'metric_name': 'Minimal Index vs Resolution Diameter', 'metric_value': nan, 'instances_tested': 1, 'conjecture': 'Minimal Index vs Resolution Diameter'
TRIAL: {'metric_name': 'Minimal Index vs Resolution Diameter', 'metric_value': nan, 'instances_tested': 1, 'conjecture': 'Minimal Index vs Resolution Diameter'
TRIAL: {'metric_name': 'Minimal Index vs Resolution Diameter', 'metric_value': nan, 'instances_tested': 1, 'conjecture': 'Minimal Index vs Resolution Diameter'
TRIAL: {'metric_name': 'Minimal Index vs Resolution Diameter', 'metric_value': nan, 'instances_tested': 1, 'conjecture': 'Minimal Index vs Resolution Diameter'
TRIAL: {'metric_name': 'Minimal Index vs Resolution Diameter', 'metric_value': nan, 'instances_tested': 1, 'conjecture': 'Minimal Index vs Resolution Diameter'
TRIAL: {'metric_name': 'Minimal Index vs Resolution Diameter', 'metric_value': nan, 'instances_tested': 1, 'conjecture': 'Minimal Index vs Resolution Diameter'
TRIAL: {'metric_name': 'Minimal Index vs Resolution Diameter', 'metric_value': nan, 'instances_tested': 1, 'conjecture': 'Minimal Index vs Resolution Diameter'
TRIAL: {'metric_name': 'Minimal Index vs Resolution Diameter', 'metric_value': nan, 'instances_tested': 1, 'conjecture': 'Minimal Index vs Resolution Diameter'
RESULT: FALSIFIED counterexample="mapping_undefined" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test has only one instance tested, which is insufficient to draw a conclusion about the conjecture. Additionally, the error message ‘too many values to unpack (expected 2)’ suggests a potential bug in the metric definition or data processing, which could lead to incorrect results.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test has produced a counterexample with the message ‘too many values to unpack (expected 2)’, indicating that the conjecture does not hold for at | next: Investigate the error message ‘too many values to unpack (expected 2)’ to identify and fix the bug in the metric definition or data processing. Once the issue is resolved, retest the conjecture with a larger sample size to confirm the result.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11258
2	preregistration	ollama_remote	glm4:latest	0	0	5308
3	novelty	ollama_remote	glm4:latest	0	0	4646
4	novelty	ollama_remote	glm4:latest	0	0	7093
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13714
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10816
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12367
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12762
9	critic	ollama_remote	glm4:latest	0	0	12152

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	judge	ollama_remote	glm4:latest	0	0	8596

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98711 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/2d8adf396e64.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2d8adf396e64.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2d8adf396e64.tar.gz (if generated)